



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

PHÂN TÍCH DỮ LIỆU (Data Analytics)

CHƯƠNG 2 : GIỚI THIỆU DỮ LIỆU LỚN (BIG DATA)



GVGD: PHAN HỒ VIỆT TRƯỜNG



NỘI DUNG

01. GIỚI THIỆU

02. CÁC MẪU DỮ LIỆU LỚN

03. CÁC MẪU MAP REDUCE

04. BÀI TẬP



Giới thiệu dữ liệu lớn

Trong một phút,

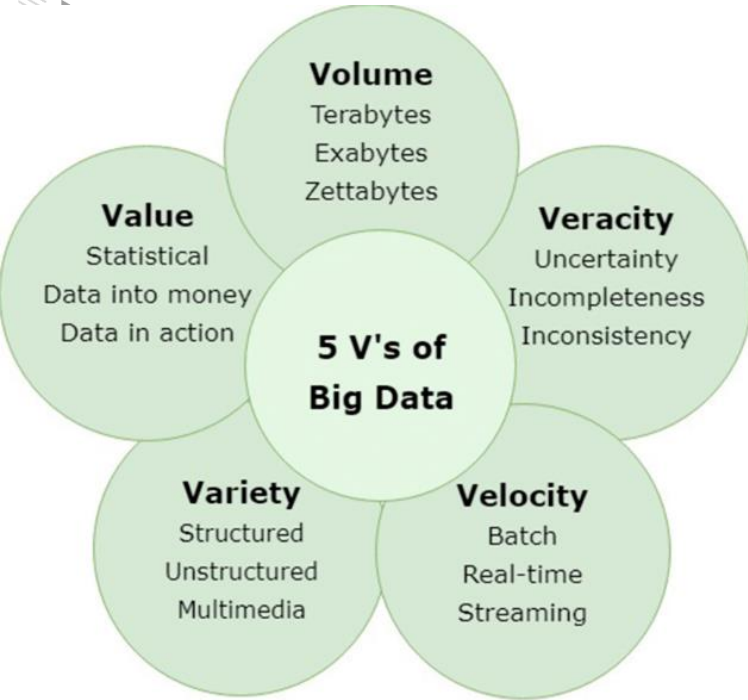
- Facebook users share nearly 4.16 million pieces of content
- Twitter users send nearly 300,000 tweets
- Instagram users like nearly 1.73 million photos
- YouTube users upload 300 hours of new video content
- Apple users download nearly 51,000 apps
- Skype users make nearly 110,000 new calls
- Amazon receives 4300 new visitors
- Uber passengers take 694 rides
- Netflix subscribers stream nearly 77,000 hours of video



Giới thiệu dữ liệu lớn

- Dữ liệu lớn (big data) là tập hợp của các bộ dữ liệu có khối lượng (volume), tốc độ (velocity), hoặc cấu trúc khác nhau (variety)
- Không thể lưu trữ, quản lý, và phân tích bằng các cơ sở dữ liệu truyền thống.

Giới thiệu dữ liệu lớn



- **Volume** (Khối lượng): Lượng dữ liệu khổng lồ, được đo bằng terabyte, petabyte, thậm chí exabyte.
- **Velocity** (Tốc độ): Dữ liệu được tạo ra và xử lý với tốc độ cực nhanh, đòi hỏi khả năng xử lý thời gian thực.
- **Variety** (Sự đa dạng): Dữ liệu đến từ nhiều nguồn khác nhau, bao gồm văn bản, hình ảnh, video, âm thanh, dữ liệu cảm biến, v.v.
- **Veracity** (Độ tin cậy): Dữ liệu có thể chứa nhiều thông tin không chính xác, thiếu sót hoặc không nhất quán, cần được kiểm tra và xử lý cẩn thận.
- **Value** (Giá trị): Dữ liệu có tiềm năng mang lại giá trị to lớn cho doanh nghiệp, tổ chức và cá nhân.

Ứng dụng của dữ liệu lớn





Thách thức với dữ liệu lớn

- Xử lý dữ liệu: Xử lý lượng dữ liệu khổng lồ đòi hỏi những công nghệ và kỹ năng chuyên biệt.
- Lưu trữ dữ liệu: Lưu trữ dữ liệu lớn cần những hệ thống lưu trữ hiệu quả, bảo mật và có khả năng mở rộng.
- Bảo mật dữ liệu: Bảo mật dữ liệu cá nhân là vấn đề quan trọng cần được ưu tiên hàng đầu.
- Phân tích dữ liệu: Phân tích dữ liệu lớn đòi hỏi những kỹ năng chuyên môn, những công cụ phân tích mạnh mẽ và khả năng xử lý dữ liệu hiệu quả.
- Hiểu rõ dữ liệu: Hiểu rõ ý nghĩa của dữ liệu, phân biệt thông tin hữu ích và thông tin không cần thiết là điều quan trọng để đưa ra quyết định chính xác.

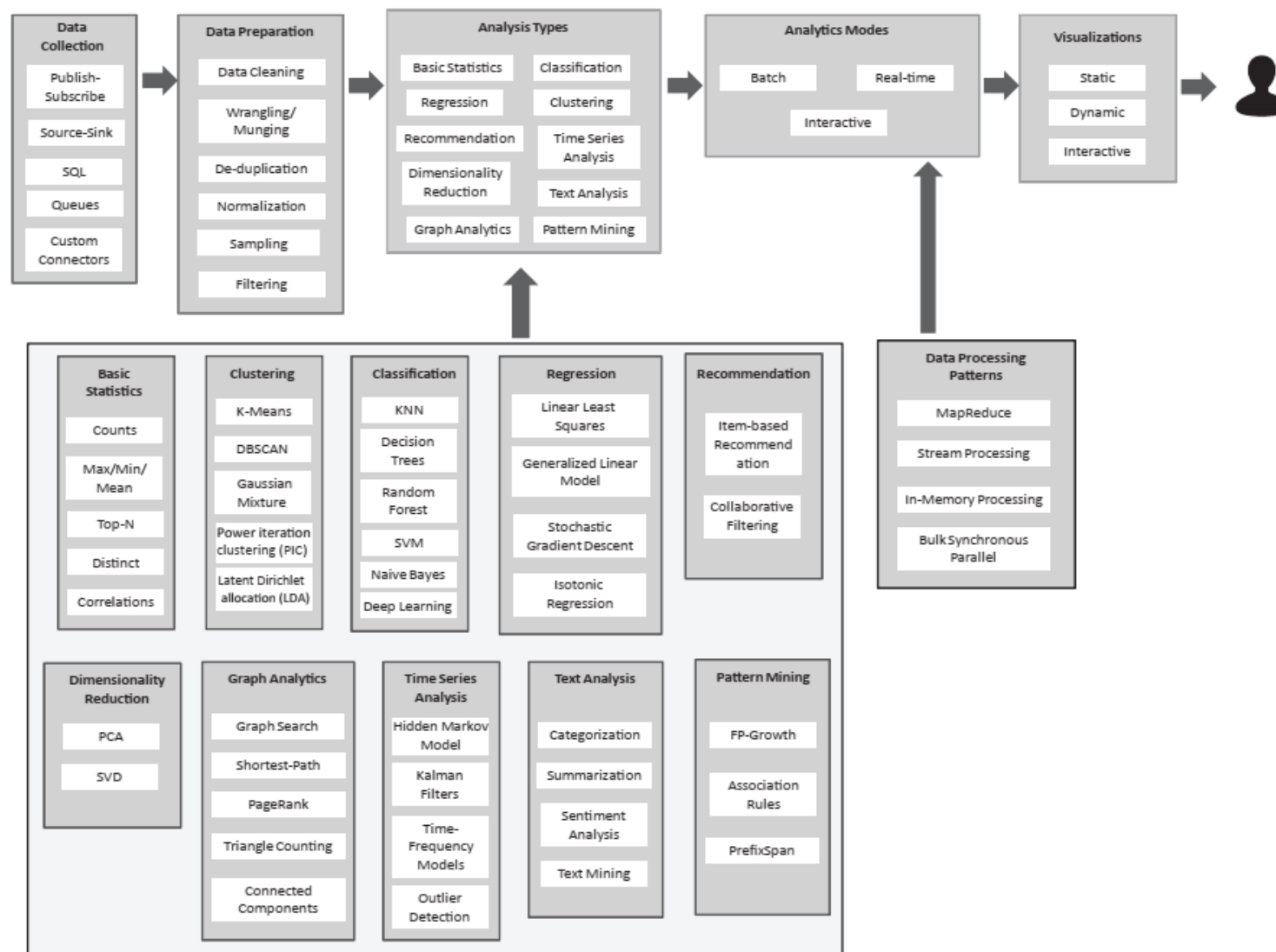




Hình thức phân tích dữ liệu lớn

- Real time mode: ứng dụng yêu cầu cập nhật sau khoảng thời gian ngắn.
- Batch mode: ứng dụng sinh kết quả và cập nhật hàng ngày hoặc hàng tháng
- Interactive mode: Ứng dụng yêu cầu có sự linh hoạt trong truy vấn dữ liệu.

Quy trình phân tích dữ liệu lớn



Quy trình phân tích dữ liệu lớn

Data Collection

Analysis Type	Framework (Mode)
Publish-Subscribe	Kafka, Kinesis
Source-Sink	Flume
SQL	Sqoop
Queues	SQS, RabbitMQ, ZeroMQ, RESTMQ
Custom Connectors	REST, WebSocket, MQTT

Data Preparation

Analysis Type	Framework
Data Cleaning	Open Refine
Data Wrangling	Open Refine DataWrangler
De-Duplication	Open Refine, Pig, Hive, Spark SQL
Normalization Sampling, Filtering	MapReduce, Pig, Hive, Spark SQL

Basic Statistics

Analysis Type	Framework (Mode)
Counts, Max, Min, Mean, Top-N, Distinct	Hadoop-MapReduce (Batch), Pig (Batch), Spark (Batch), Spark Streaming (Realtime), Spark SQL (Interactive), Hive (Integrative), Storm (Real-time)
Correlations	Hadoop-MapReduce (Batch), Spark Mlib (Batch)

Clustering

Analysis Type	Framework (Mode)
K-Means	Hadoop-MapReduce (Batch), Spark Mlib (Batch & Real-time) H2O (Batch)
DBSCAN	Spark (Batch)
Gaussian Mixture	Spark Mlib (Batch)
PIC	Spark Mlib (Batch)
LDA	Spark Mlib (Batch)

Classification

Analysis Type	Framework (Mode)
KNN	Spark Mlib (Batch , Realtime)
Decision Trees	Spark Mlib (Batch, Realtime)
Random Forest	Spark Mlib (Batch , Realtime), H2O (Batch)
SVM	Spark Mlib (Batch , Realtime)
Naïve Bayes	Spark Mlib (Batch, Realtime), H2O (Batch)
Deep Learning	H2O (Batch)

Regression

Analysis Type	Framework (Mode)
Linear Least Squares	Spark Mlib (Batch, Realtime)
Generalized Linear Model	H2O (Batch)
Stochastic Gradient Descent	Spark Mlib (Batch, Realtime)
Isotonic Regression	Spark Mlib (Batch, Realtime)

Quy trình phân tích dữ liệu lớn

Graph Analytics

Analysis Type	Framework (Mode)
Graph Search	Spark GraphX (Batch)
Shortest-Path	Spark GraphX (Batch)
PageRank	Spark GraphX (Batch)
Triangle Counting	Spark GraphX (Batch)
Connected Components	Spark GraphX (Batch)

Time Series Analysis

Analysis Type	Framework (Mode)
Kalman Filter	Spark (Realtime)
Time Frequency Models	Spark (Realtime)
Outlier Detection	Storm (Realtime), Spark (Batch, Realtime)

Dimensionality Reduction

Analysis Type	Framework (Mode)
SVD	Spark Mlib (Batch)
PCA	Spark Mlib (Batch), H2O (Batch)

Recommendation

Analysis Type	Framework (Mode)
Item-bases Recommendation	Spark Mlib (Batch)
Collaborative Filtering	Spark Mlib (Batch)

Text Analysis

Analysis Type	Framework (Mode)
Categorization	Hadoop-MapReduce (Batch), Storm (Realtime), Spark (Batch, Realtime)
Summarization	Spark (Batch)
Sentiment Analysis	Storm (Realtime), Spark (Batch, Realtime)
Text Mining	Storm (Realtime), Spark (Batch, Realtime)

Pattern Mining

Analysis Type	Framework (Mode)
FP-Growth	Spark Mlib (Batch)
Association Rules	Spark Mlib (Batch)
PrefixSpan	Spark Mlib (Batch)

Visualization

Analysis Type	Framework (Mode)
Web Frameworks	Django, Flask
SQL Databases	MySQL
NoSQL Databases	Hbase, DynamoDB, Cassandra, MongoDB
Visualization Frameworks	Lightning, pyGal, Seaborn



Bài tập

1. Tình huống phân tích dữ liệu khách hàng trong bán lẻ

- Công ty bán lẻ X đang thu thập dữ liệu từ nhiều kênh như bán hàng trực tuyến, tại cửa hàng, và các tương tác trên mạng xã hội. Làm thế nào để công ty này có thể tích hợp và phân tích dữ liệu lớn từ các nguồn khác nhau để cải thiện trải nghiệm khách hàng và tăng doanh số bán hàng?
- Thảo luận các bước thu thập, làm sạch và phân tích dữ liệu.
- Sử dụng mô hình phân tích dự đoán (predictive analytics) để dự đoán hành vi mua hàng trong tương lai.
- Các công cụ như Hadoop, Spark và các giải pháp kho dữ liệu đám mây có thể được ứng dụng như thế nào?



Bài tập

2. Tình huống phân tích dữ liệu IoT trong sản xuất

- Một nhà máy sản xuất có hệ thống cảm biến IoT trên các máy móc để theo dõi hiệu suất và tình trạng thiết bị theo thời gian thực. Làm thế nào để doanh nghiệp sử dụng phân tích dữ liệu lớn để dự đoán bảo trì thiết bị trước khi xảy ra sự cố?
- Thảo luận về quy trình thu thập dữ liệu từ IoT.
- Làm sạch và chuẩn hóa dữ liệu từ nhiều loại cảm biến khác nhau.
- Ứng dụng các kỹ thuật phân tích thời gian thực để dự đoán và lên lịch bảo trì.



Bài tập

3. Tình huống phân tích dữ liệu y tế

- Một bệnh viện muốn phân tích dữ liệu bệnh nhân để tối ưu hóa quy trình điều trị và quản lý nguồn lực y tế. Họ đang thu thập dữ liệu từ hệ thống hồ sơ bệnh nhân điện tử (EMR), cảm biến sức khỏe, và các phòng xét nghiệm. Làm thế nào để bệnh viện áp dụng phân tích dữ liệu lớn để cải thiện chất lượng chăm sóc sức khỏe?
- Thảo luận về cách thu thập và lưu trữ dữ liệu y tế một cách an toàn.
- Tích hợp và phân tích dữ liệu từ các nguồn khác nhau để tối ưu hóa các phác đồ điều trị.
- Phân tích xu hướng bệnh tật để dự đoán nhu cầu sử dụng nguồn lực y tế trong tương lai.



Bài tập

4. Tình huống phân tích dữ liệu giao thông đô thị

- Một thành phố đang tìm cách tối ưu hóa hệ thống giao thông bằng cách sử dụng dữ liệu từ các camera, cảm biến đường phố, và thiết bị định vị GPS trên các phương tiện công cộng. Làm thế nào để thành phố này áp dụng phân tích dữ liệu lớn để giảm thiểu tắc nghẽn giao thông và tối ưu hóa thời gian di chuyển?
- Thảo luận quy trình thu thập dữ liệu từ nhiều nguồn giao thông khác nhau.
- Ứng dụng phân tích dữ liệu thời gian thực để điều chỉnh tín hiệu đèn giao thông và phân bổ lại nguồn lực giao thông.
- Phân tích dữ liệu lịch sử để dự đoán và ngăn chặn các điểm tắc nghẽn tiềm năng.

PHÂN TÍCH DỮ LIỆU

(Data Analytics)

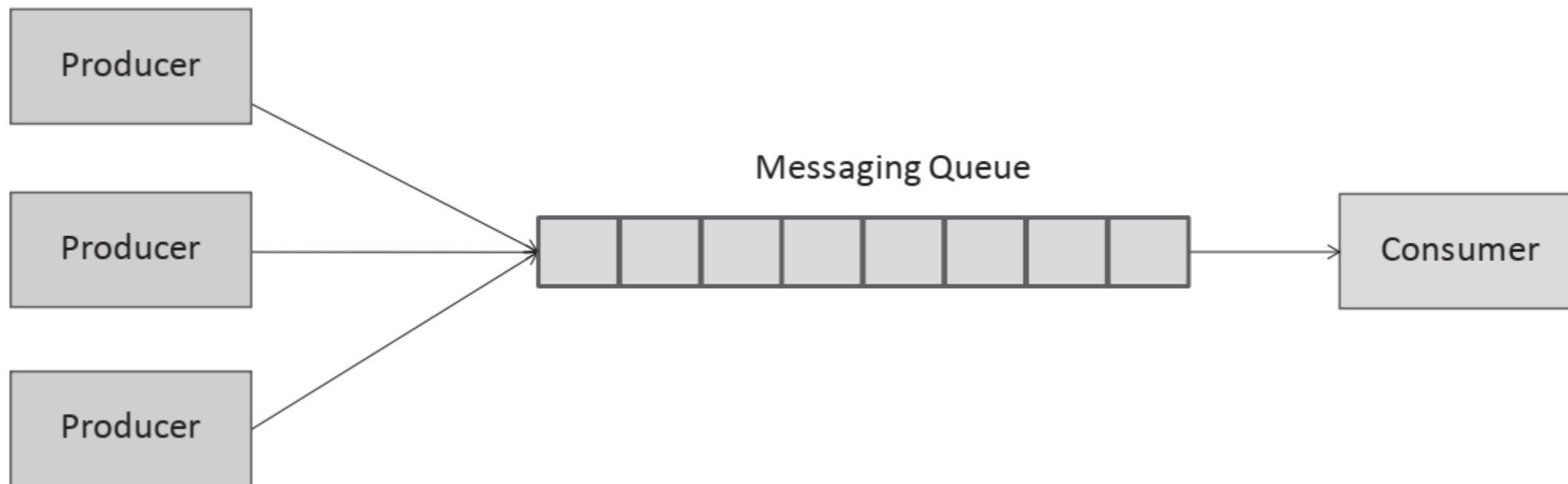
CHƯƠNG 3 : MẪU DỮ LIỆU LỚN



GVGD: PHAN HỒ VIỆT TRƯỜNG

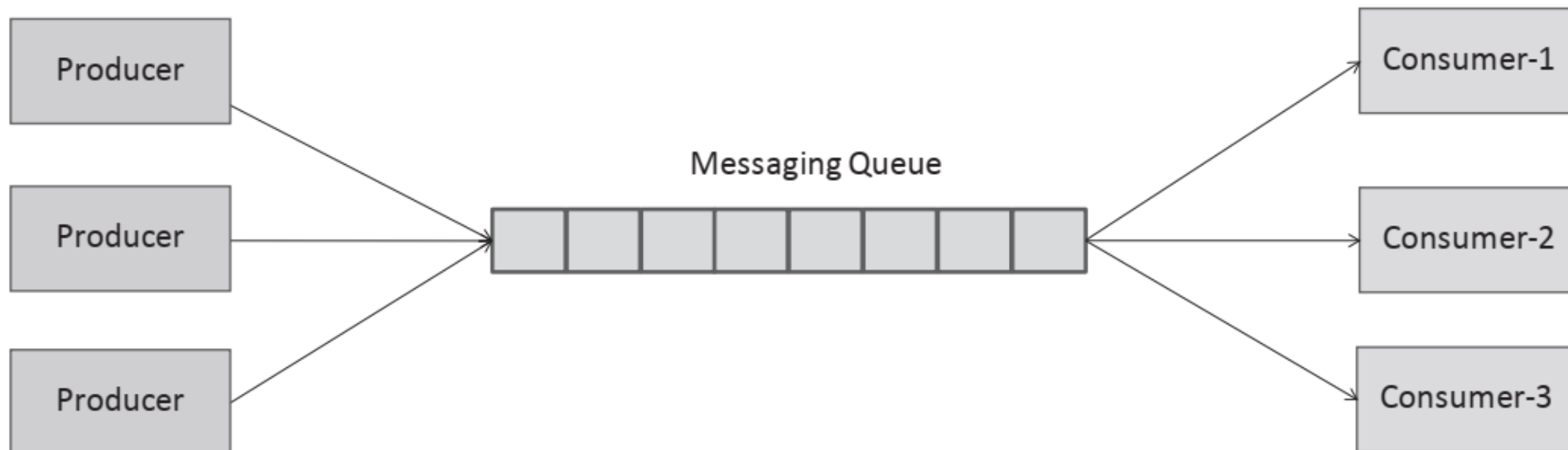
Thiết kế & thành phần kiến trúc phân tích

- Cân bằng tải (load leveling) với hàng đợi
- Hàng đợi thông điệp là trung gian tiếp nhận dữ liệu từ data producer và cung cấp dữ liệu cho data consumer.



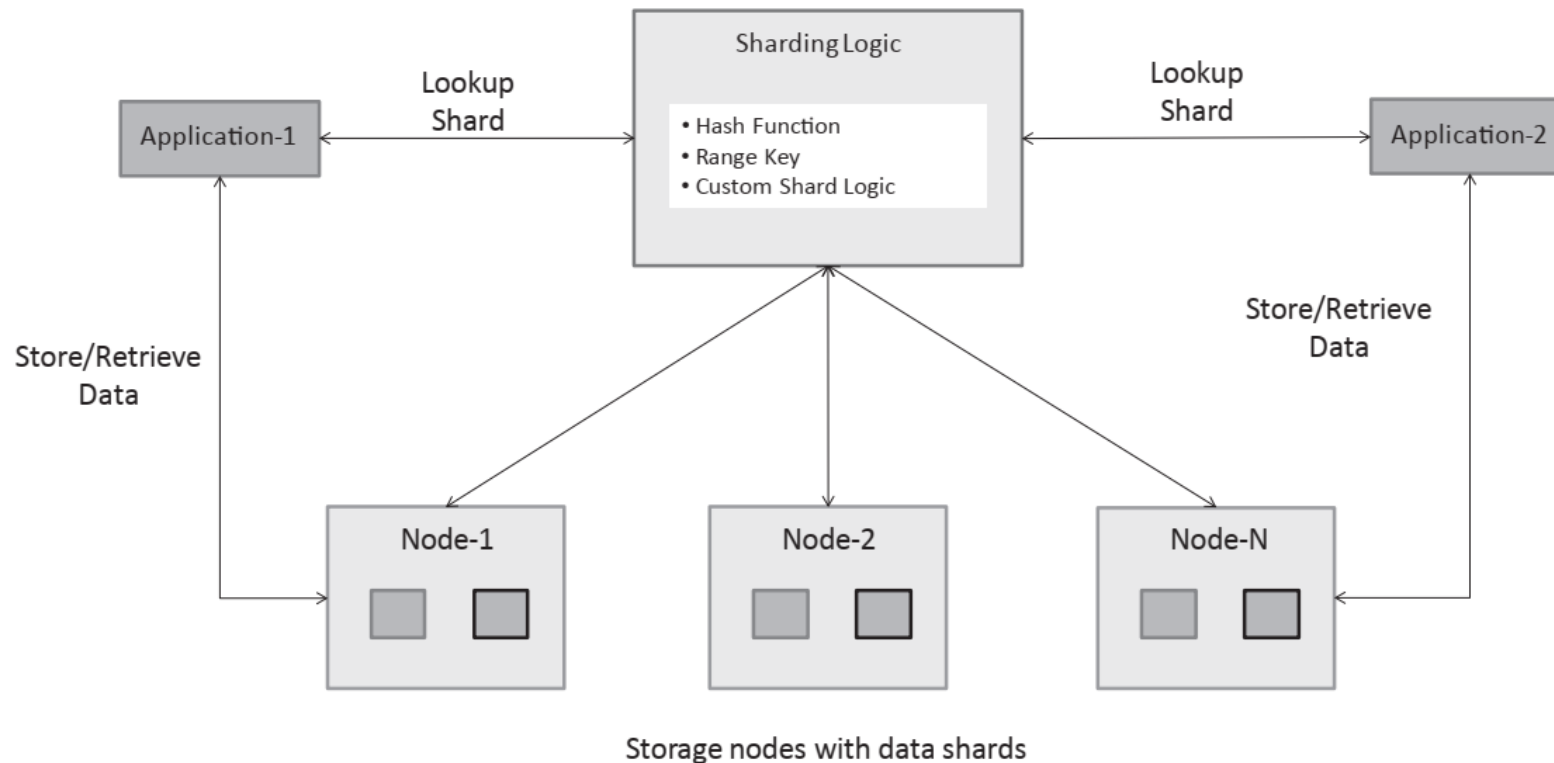
Thiết kế & thành phần kiến trúc phân tích

- Cân bằng tải (load leveling) với nhiều consumers
- Có nhiều data consumer giúp cân bằng và làm cho hệ thống xử lý cấp độ lớn hơn (song song), tin cậy hơn và luôn có sẵn.
- Thông điệp chỉ cấp riêng cho một consumer tại 1 thời điểm.



Thiết kế & thành phần kiến trúc phân tích Sharding

- Phân hoạch ngang dữ liệu lưu trữ trên nhiều node trong hệ thống.
- Sharding sử dụng một hoặc nhiều cột data để làm key (dùng bảng băm).



Shards may be replicated on the storage nodes



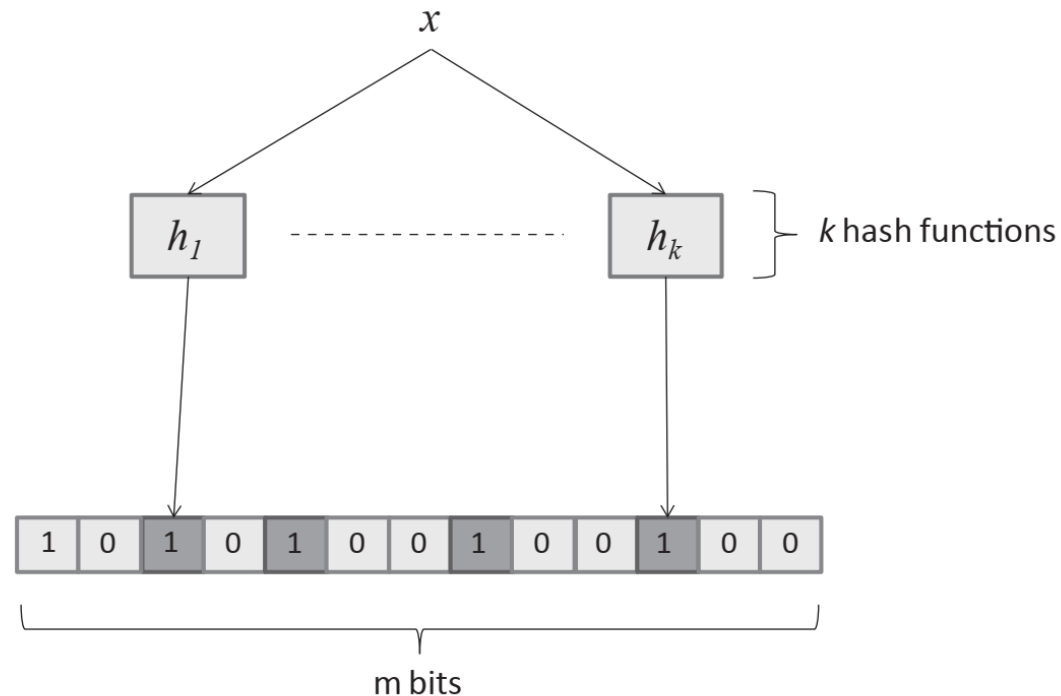
Thiết kế & thành phần kiến trúc phân tích

Consistency, Availability, Partition Tolerance

- CAP cho rằng khi phân hoạch dữ liệu thì hệ thống phân tán chỉ có thể thực hiện consistency hoặc availability và không đồng thời cả hai.
- Hệ thống nhất quán đảm bảo việc đọc dữ liệu kết hợp với việc ghi trước đó. Sau khi 1 tác vụ cập nhật bởi bộ ghi thì bộ đọc có thể đọc được dữ liệu.
- Hệ thống khả dụng có thể trả lời tất cả câu truy vấn mà không bị gián đoạn.
- Khả năng chịu tải phân vùng mạng là khả năng thực hiện liên tục các phép toán kể cả trong trường hợp 1 vài node bị lỗi.

Thiết kế & thành phần kiến trúc phân tích Bộ lọc Bloom

- Bloom filter là một cấu trúc mảng có m bits và dùng để kiểm tra một phần tử có thuộc một tập hợp hay không.
- Khi thêm 1 phần tử vào tập hợp, hàm băm k ánh xạ phần tử vào vị trí trong mảng bit và được gán giá trị 1.



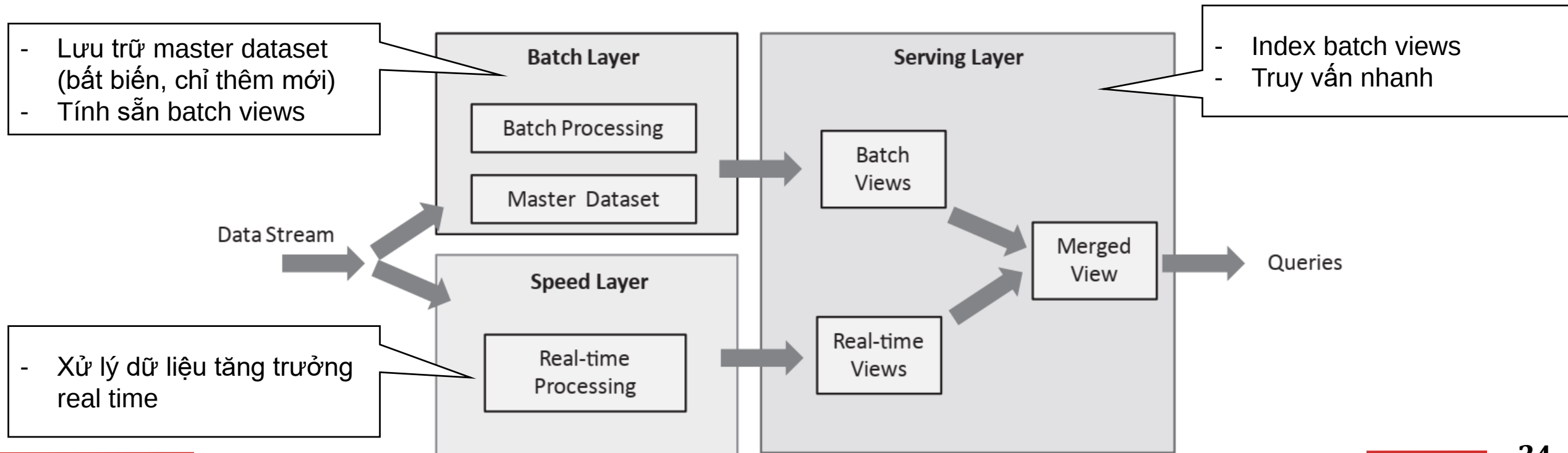


Thiết kế & thành phần kiến trúc phân tích Materialized Views

- Là việc tính toán trước các truy vấn thường xuyên được thực hiện nhằm cải tiến thời gian phản hồi.
- Những truy vấn phức tạp (tổng hợp trên nhiều bảng...) được tính sẵn và lưu vào ổ đĩa hoặc cache.
- Đặc biệt có lợi trên hệ thống phân tán dữ liệu.

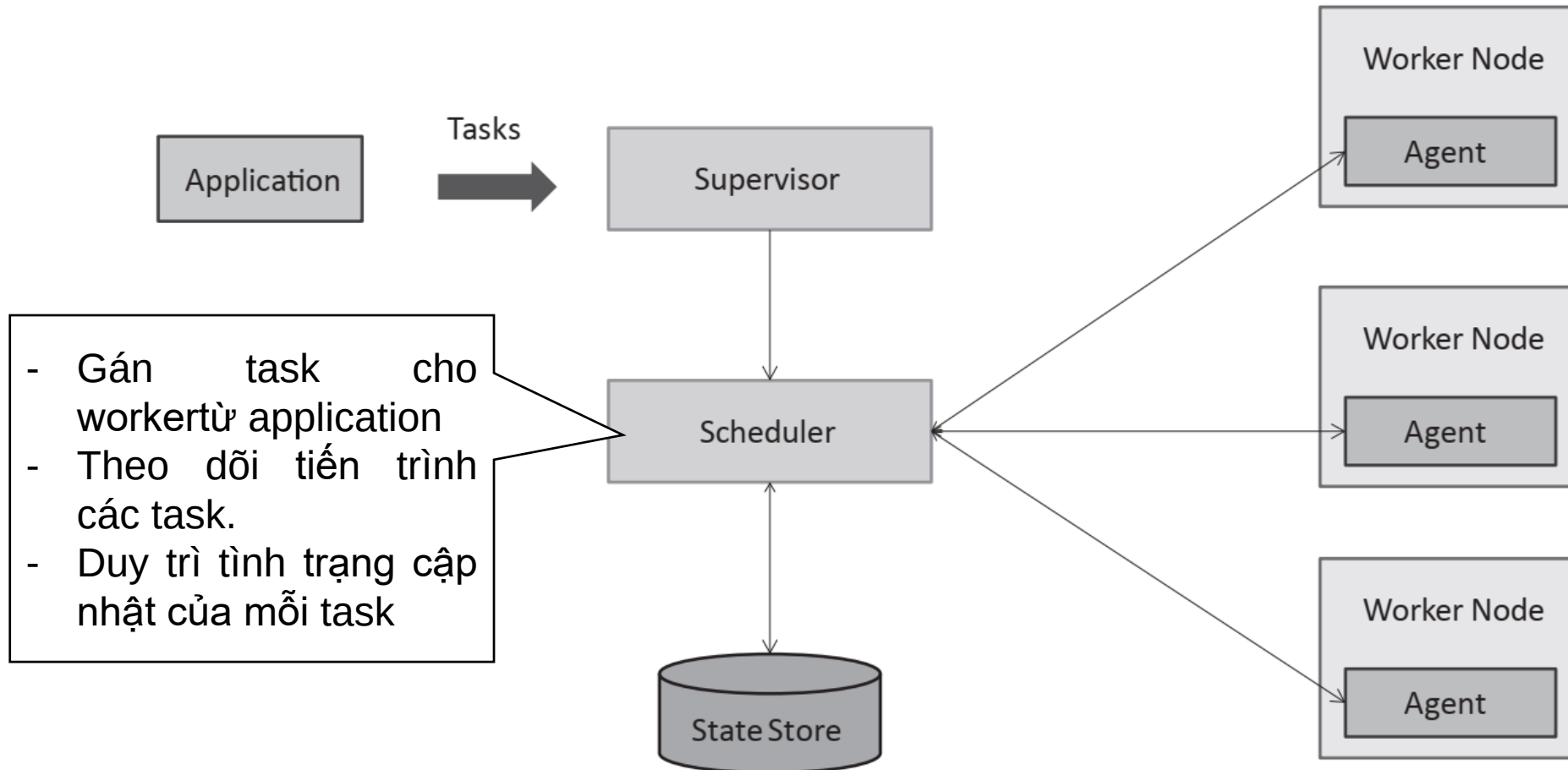
Thiết kế & thành phần kiến trúc phân tích Kiến trúc Lambda

- Là kiến trúc xử lý dữ liệu có khả năng mở rộng và khả năng chịu lỗi cho các luồng dữ liệu đến liên tục.
- Lambda cho phép truy vấn dữ liệu lịch sử và real time.



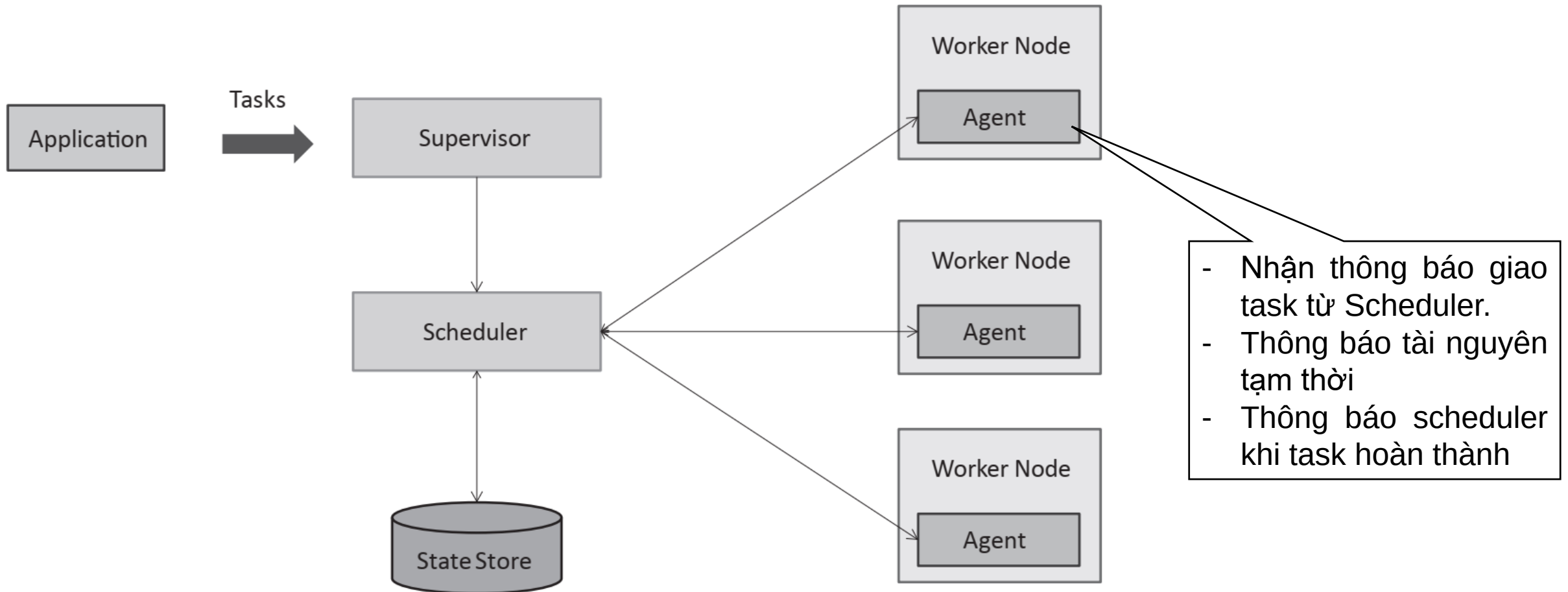
Thiết kế & thành phần kiến trúc phân tích Scheduler-Agent-Supervisor

- Đảm bảo các worker có thể phục hồi và chịu lỗi



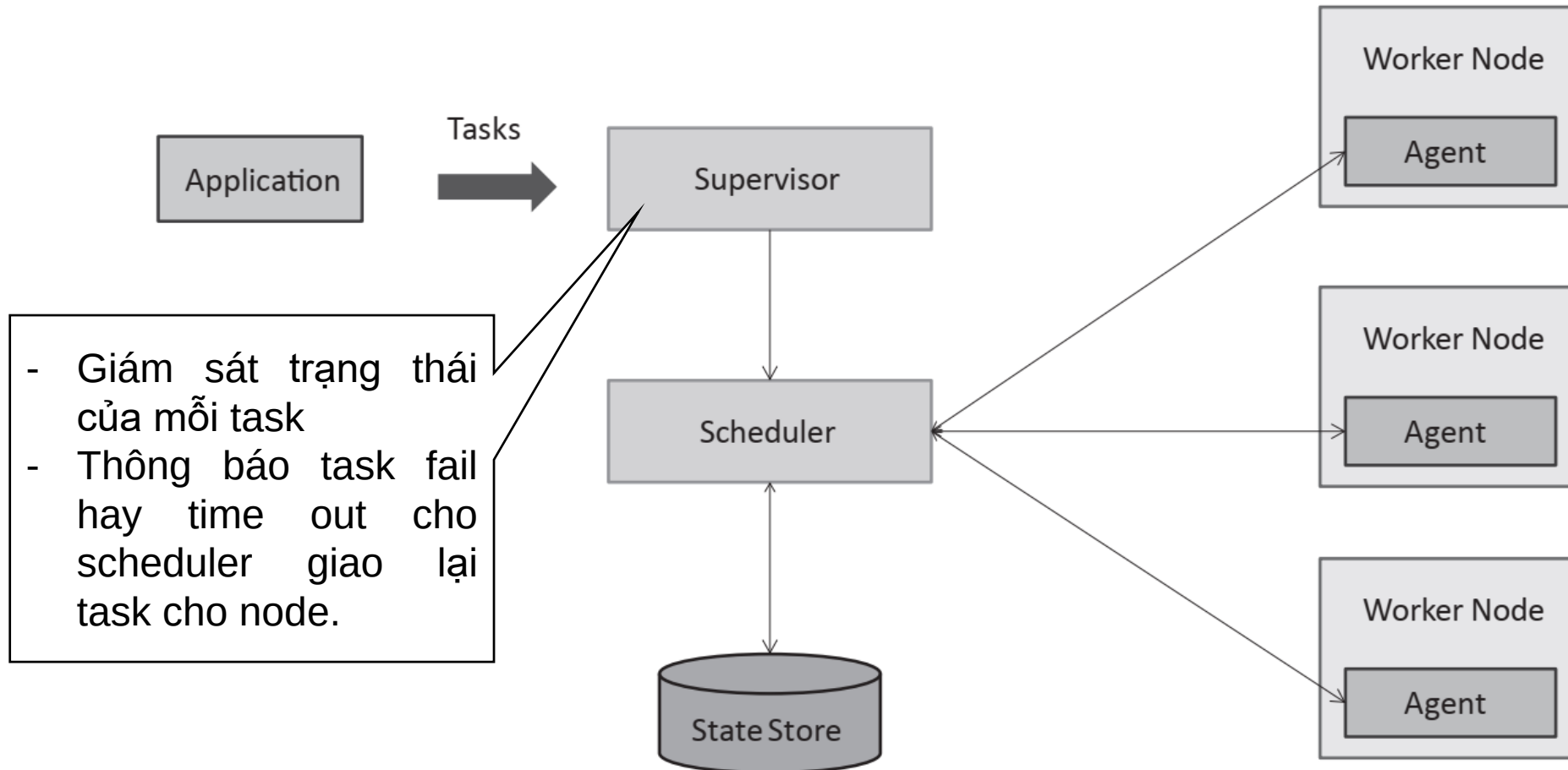
Thiết kế & thành phần kiến trúc phân tích Scheduler-Agent-Supervisor

- Đảm bảo các worker có thể phục hồi và chịu lỗi

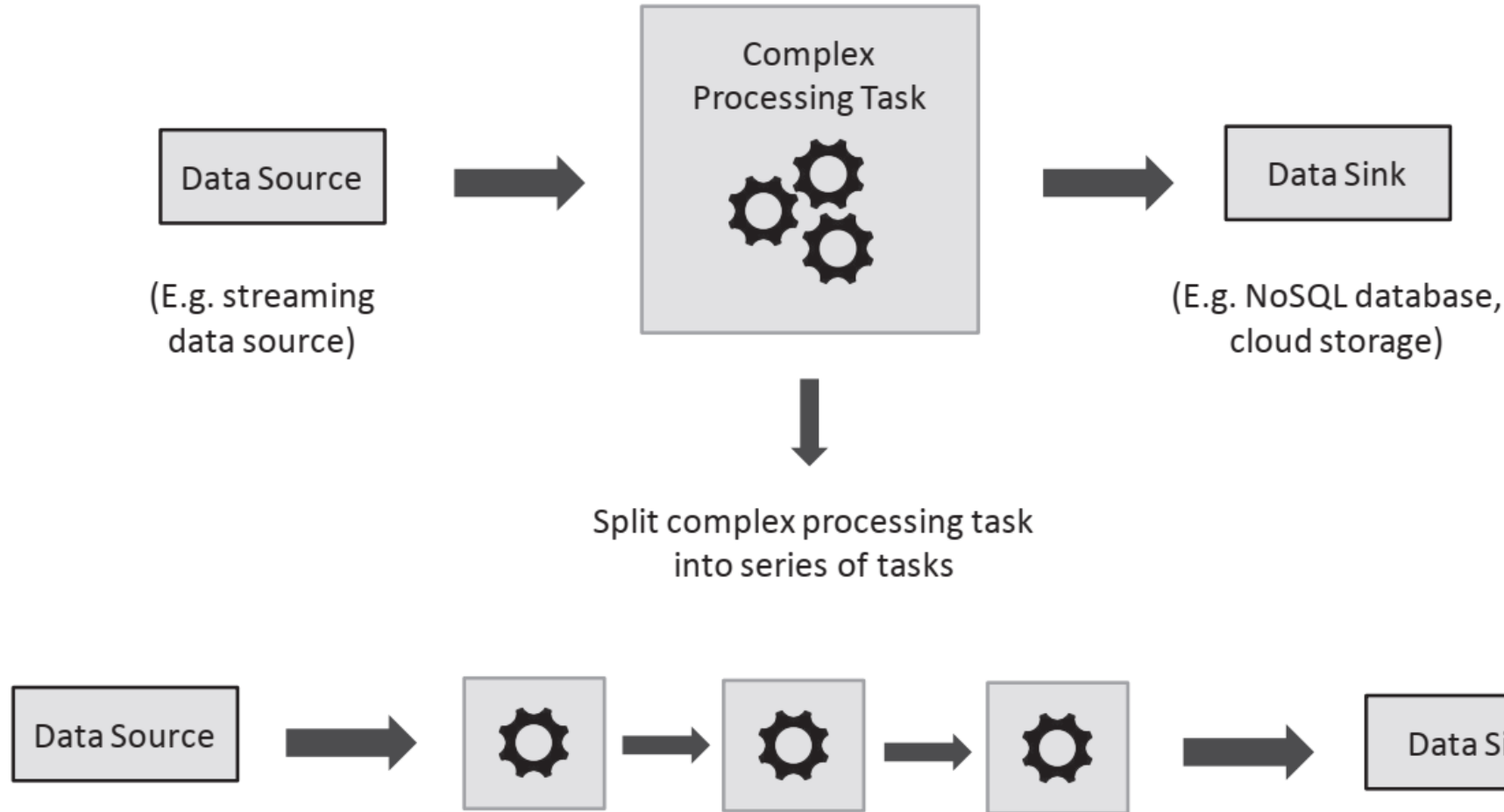


Thiết kế & thành phần kiến trúc phân tích Scheduler-Agent-Supervisor

- Đảm bảo các worker có thể phục hồi và chịu lỗi

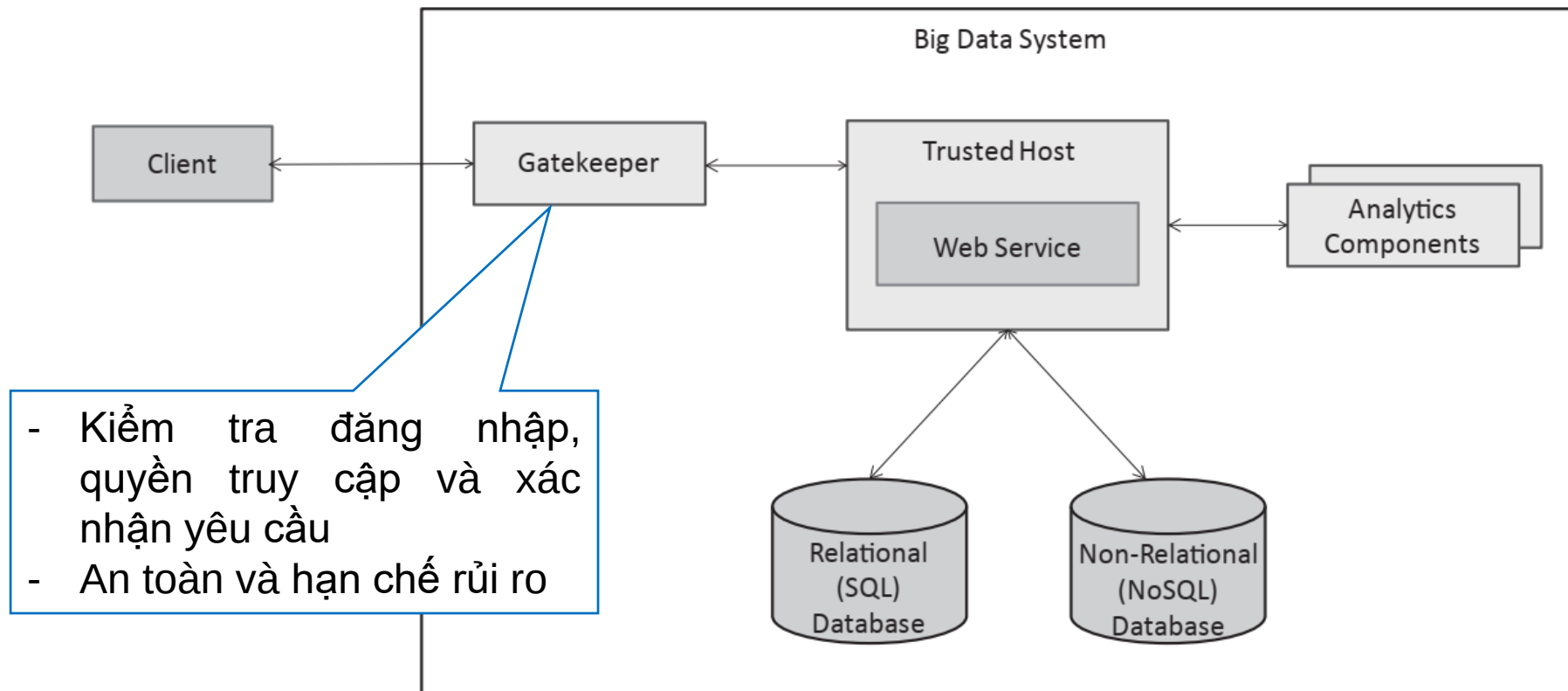


Thiết kế & thành phần kiến trúc phân tích Pipes & Filters



Thiết kế & thành phần kiến trúc phân tích Web service

- Giao diện cung cấp các chức năng của hệ thống để máy khách nhận những kết quả dưới nhiều hình thức khác nhau





Thiết kế & thành phần kiến trúc phân tích

Bài tập

1. Thảo luận về các phương pháp thu thập dữ liệu từ nhiều nguồn khác nhau (cơ sở dữ liệu, logs, streams). Các hệ thống như Apache Kafka, Flume, hay Kinesis đóng vai trò gì trong kiến trúc này?
2. So sánh các hệ thống lưu trữ như Hadoop Distributed File System (HDFS), Amazon S3, và Google Cloud Storage. Khi nào nên chọn hệ thống lưu trữ phân tán so với cơ sở dữ liệu truyền thống?
3. Thảo luận về kiến trúc xử lý dữ liệu theo lô với Apache Hadoop và Spark. Ưu nhược điểm của xử lý theo lô so với xử lý theo thời gian thực là gì?
4. So sánh giữa Apache Storm, Flink, và Spark Streaming. Khi nào nên sử dụng xử lý thời gian thực và lợi ích của việc này trong Big Data là gì?

Thiết kế & thành phần kiến trúc phân tích

Bài tập

5. Phân tích vai trò của Apache Spark trong việc xử lý dữ liệu trong bộ nhớ (in-memory processing). Khi nào điều này có lợi hơn so với việc lưu trữ và xử lý trên đĩa cứng?
6. Thảo luận về các hệ thống truy vấn như Apache Hive, Presto, và Google BigQuery. So sánh hiệu suất và trường hợp sử dụng của mỗi hệ thống.
7. Apache Airflow và Oozie đóng vai trò gì trong việc quản lý và điều phối các luồng công việc (workflow) trong Big Data? Làm thế nào để bảo đảm rằng các tiến trình hoạt động đồng bộ và hiệu quả?
8. Thảo luận về vai trò của các công cụ như Prometheus, Grafana trong việc giám sát hiệu suất của hệ thống Big Data. Làm thế nào để tối ưu hóa hiệu suất trong quá trình phân tích dữ liệu lớn?



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

PHÂN TÍCH DỮ LIỆU

(Data Analytics)

CHƯƠNG 4: CÁC MẪU MAP REDUCE



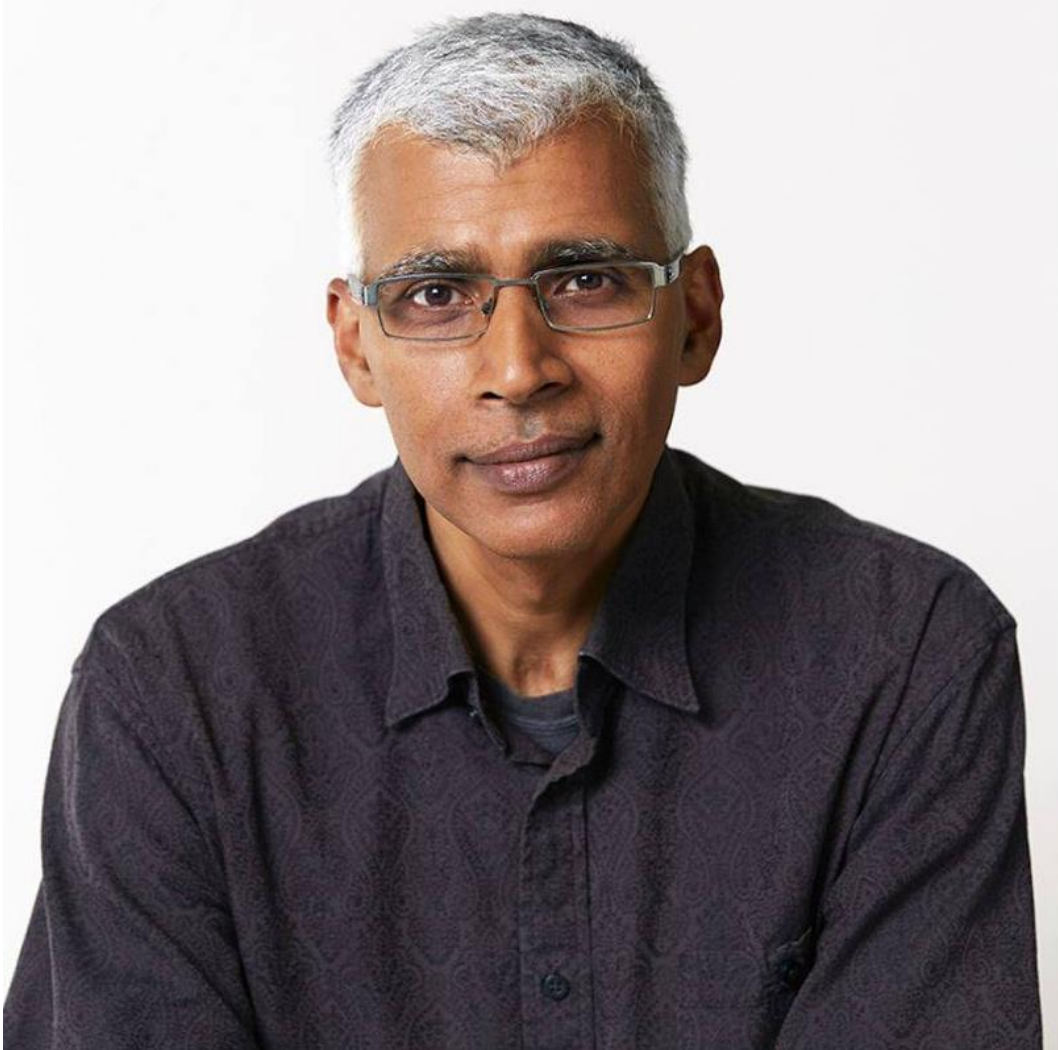
GVGD: PHAN HỒ VIỆT TRƯỜNG



Giới thiệu mô hình Map Reduce

- Dùng để phân tích dữ liệu khối trong các ứng dụng lớn (MXH, tài chính...) mà không quan tâm đến những vấn đề như tách dữ liệu đầu vào, lập lịch xử lý, cân bằng tải hay sự cố mạng.
- Mô hình lập trình Map Reduce xử lý dữ liệu trên các cụm lớn do Jeff Dean và Sanjay Ghemawat đề xuất năm 2004 (Google).
- Hadoop là hệ thống xử lý khối phân tán cỡ lớn, cài đặt cơ chế Map Reduce.

Giới thiệu mô hình Map Reduce



Sanjay Ghemawat (1966 – Mỹ Ấn)

- TS khoa học máy tính 1995
- Nghiên cứu viên cao cấp tại Google.
- Tham gia dự án Map Reduce, Tensorflow

Giới thiệu mô hình Map Reduce



Jeff Dean (23/7/1968 – Mỹ)

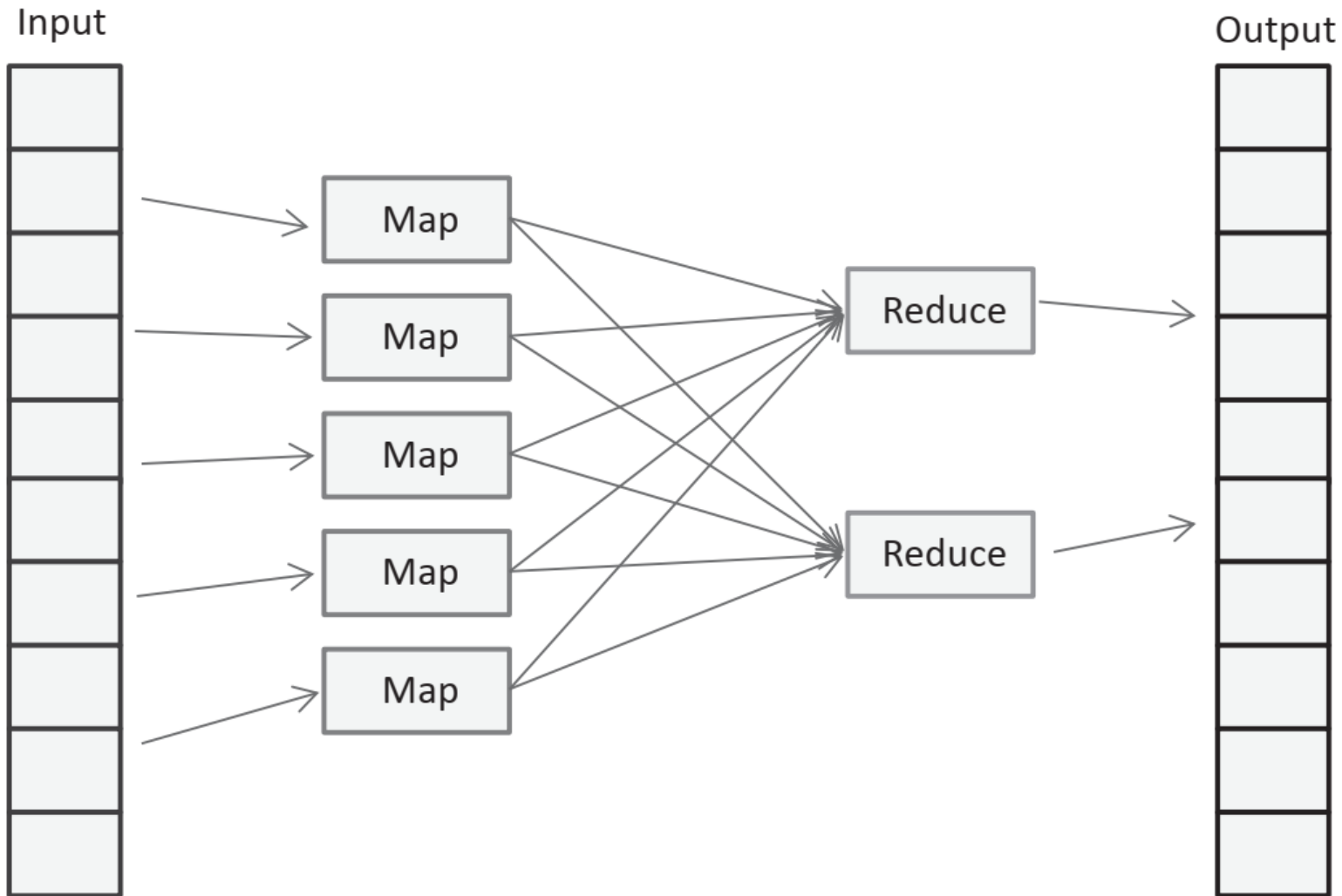
- TS khoa học máy tính 1996
- Giám đốc khoa học tại Google.
- Đồng sáng lập Google Brain, Google translate và Gemini.
- Tham gia dự án Tensorflow
- Từng đến VN tham gia sự kiện “Building AI Future for Vietnam” do DH Fullbright và Google tổ chức vào 19/8/2024
(<https://thanhvien.vn/nha-khoa-hoc-jeff-dean-sai-lam-khi-nghi-ung-dung-ai-can-rat-nhieu-tien-185240820150756843.htm>)



Giới thiệu mô hình Map Reduce

- Map Reduce gồm 2 giai đoạn: Map và Reduce
- Dữ liệu đầu vào cho Map và Reduce có dạng key-value.
- Các hệ thống run-time Map Reduce quản lý các tác vụ phân hoạch dữ liệu, lập lịch và giao tiếp giữa các node trong một cụm.

Giới thiệu mô hình Map Reduce





Mẫu tổng hợp

- Dùng để thực hiện các phép toán thống kê như count, max, min, mean ...
- Ví dụ như tính số likes cho một bài đăng cụ thể, tính lượng mưa trung bình hay số lượng khách hàng mỗi tháng trên một web site.
- Xét 1 trường hợp cụ thể, cần thống kê số lần viếng thăm trang của một Website với các thuộc tính như sau.
 - ✓ Date (yyyy-mm-dd)
 - ✓ Time (HH:MM:SS)
 - ✓ URL
 - ✓ IP
 - ✓ Visit-length



Mẫu tổng hợp

- Dùng để thực hiện các phép toán thống kê như count, max, min, mean ...
- Ví dụ như tính số likes cho một bài đăng cụ thể, tính lượng mưa trung bình hay số lượng khách hàng mỗi tháng trên một web site.
- Xét 1 trường hợp cụ thể, cần thống kê số lần viếng thăm trang của một Website với các thuộc tính như sau.
 - ✓ Date (yyyy-mm-dd)
 - ✓ Time (HH:MM:SS)
 - ✓ URL
 - ✓ IP
 - ✓ Visit-length

Mẫu tính tổng

■ Box 3.1: Python program for computing count with MapReduce

```
#Total number of times each page is visited in the year 2014

from mrjob.job import MRJob

class MRmyjob(MRJob):
    def mapper(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()

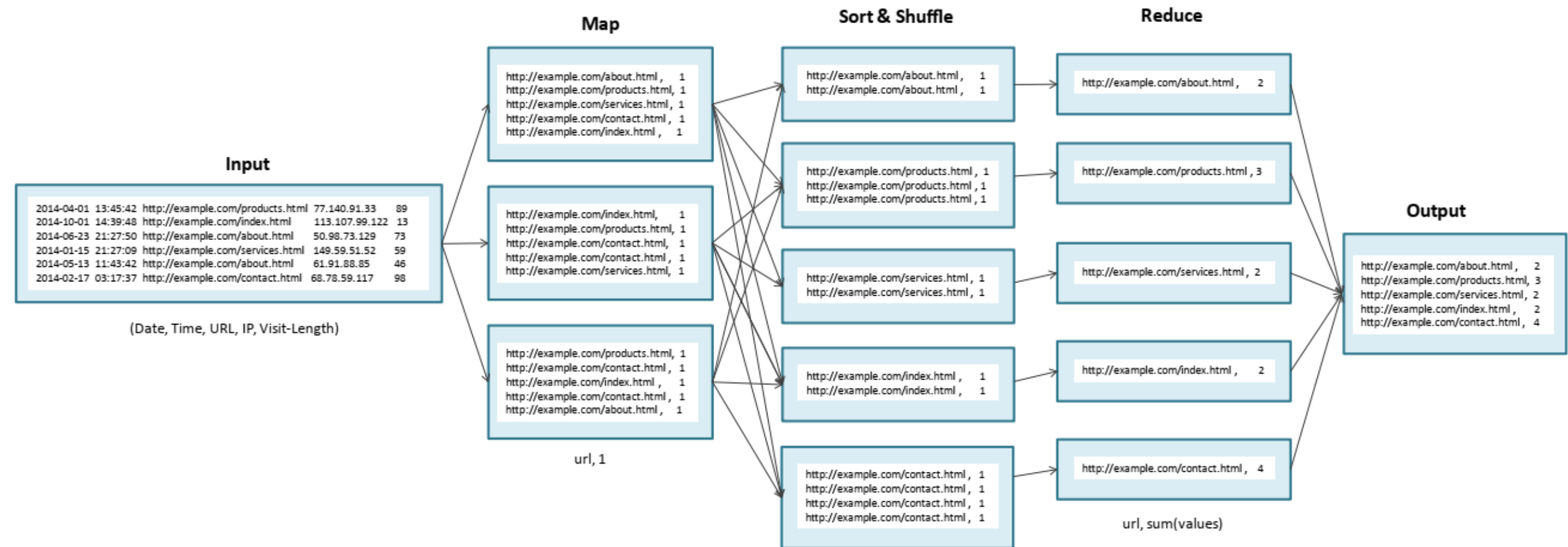
        #Extract year from date
        year=date[0:4]

        #Emit URL and 1 if year is 2014
        if year=='2014':
            yield url, 1

    def reducer(self, key, list_of_values):
        yield key,sum(list_of_values)

if __name__ == '__main__':
    MRmyjob.run()
```

Mẫu tổng hợp



Mẫu tính min/max

```
# Most visited page in each month of year 2014

from mrjob.job import MRJob

class MRmyjob(MRJob):
    def mapper1(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()

        #Extract year from date
        year=date[0:4]
        month=date[5:7]

        #Emit (month,url) pair and 1 if year is 2014
        if year=='2014':
            yield (month,url), 1
```

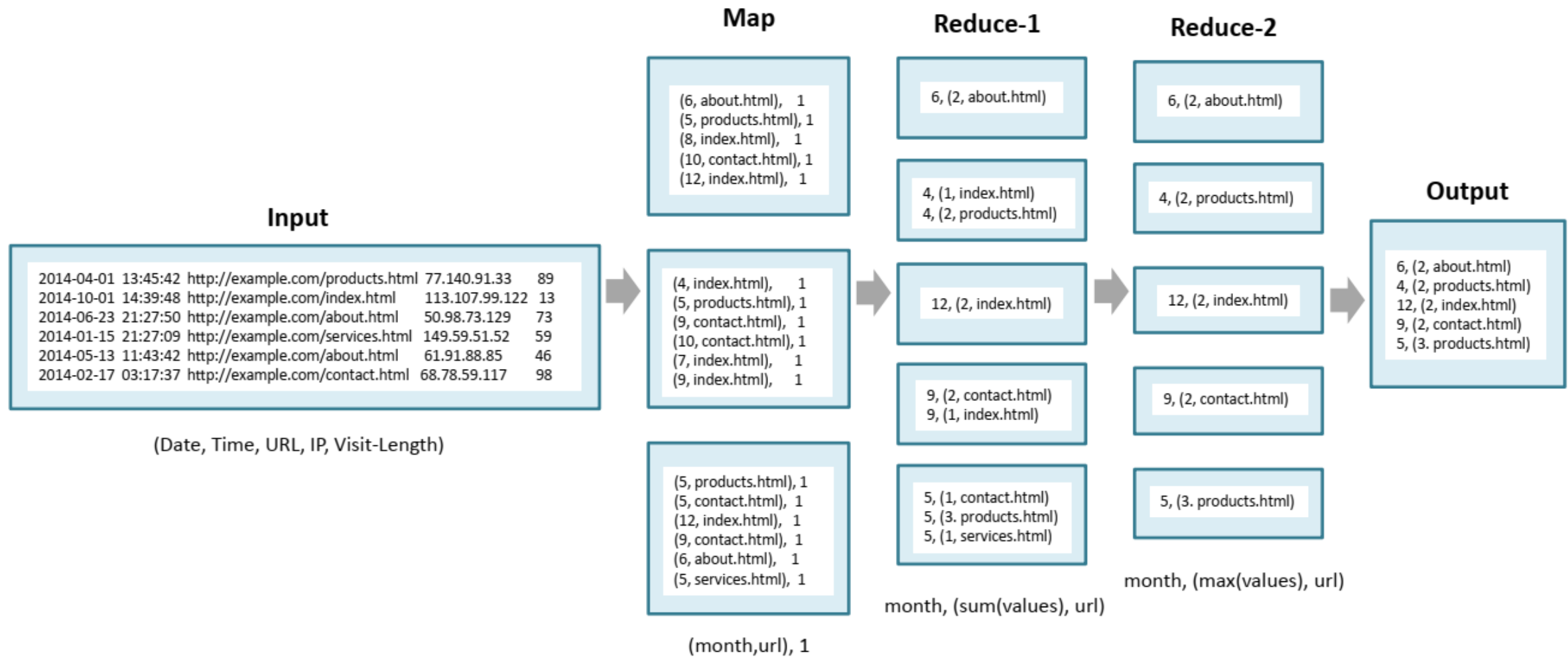
```
def reducer1(self, key, list_of_values):
    yield key[0], (sum(list_of_values), key[1])
```

```
def reducer2(self, key, list_of_values):
    yield key, max(list_of_values)
```

```
def steps(self):
    return [self.mr(mapper=self.mapper1,
                    reducer=self.reducer1), self.mr(reducer=self.reducer2)]
```

```
if __name__ == '__main__':
    MRmyjob.run()
```

Mẫu tính min/max



Mẫu tính average

#Average visit length for each page

```
from mrjob.job import MRJob
```

```
class MRmyjob(MRJob):
```

```
    def mapper(self, _, line):
```

```
        #Split the line with tab separated fields
```

```
        data=line.split('\t')
```

```
        #Parse line
```

```
        date = data[0].strip()
```

```
        time = data[1].strip()
```

```
        url = data[2].strip()
```

```
        ip = data[3].strip()
```

```
        visit_len = int(data[4].strip())
```

```
        year=date[0:4]
```

```
        month=date[5:7]
```

```
        yield url, visit_len
```

```
def reducer(self, key, list_of_values):
```

```
    count = 0
```

```
    total = 0.0
```

```
    for x in list_of_values:
```

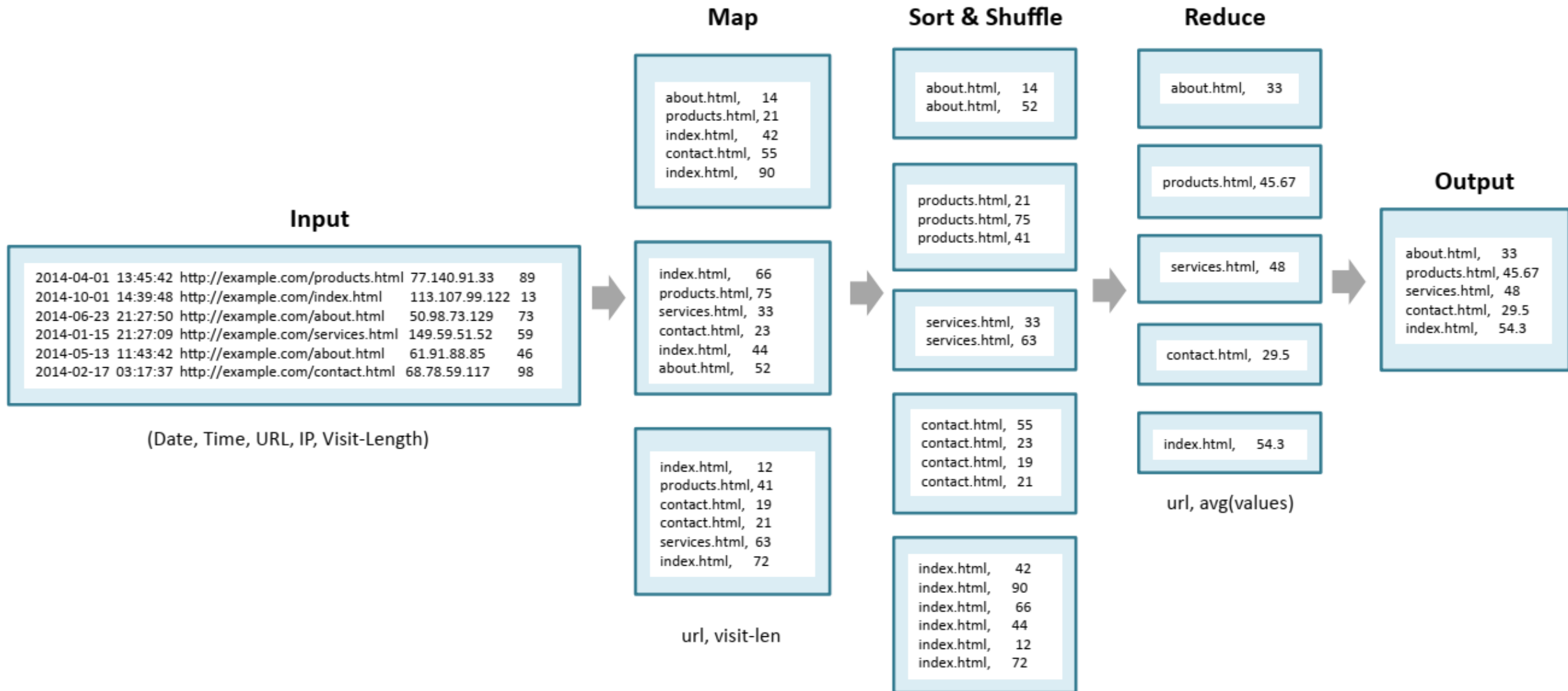
```
        total = total+x
```

```
        count=count+1
```

```
    avgLen = ("%2f" % (total/count))
```

```
    yield key, avgLen
```

Mẫu tính average



Mẫu tính N-top

```
# Top 3 visited page in each month of year 2014

from mrjob.job import MRJob

class MRmyjob(MRJob):
    def mapper(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()
        visit_len=int(data[4].strip())
        #Extract year from date
        year=date[0:4]
        month=date[5:7]

        #Emit url and 1 if year is 2014
        if year=='2014':
            yield url, 1
```

```
def reducer(self, key, list_of_values):
    total_count = sum(list_of_values)
    yield None, (total_count, key)
```

```
def reducer2(self, _, list_of_values):
    N = 3
    list_of_values = sorted(list(list_of_values), reverse=True)
    return list_of_values[:N]
```

```
def steps(self):
    return [self.mr(mapper=self.mapper1,
                    reducer=self.reducer1), self.mr(reducer=self.reducer2)]
```

Mẫu Filter

```
# Filter all page visits for the page 'contact.html' in the month of Dec
2014.

from mrjob.job import MRJob

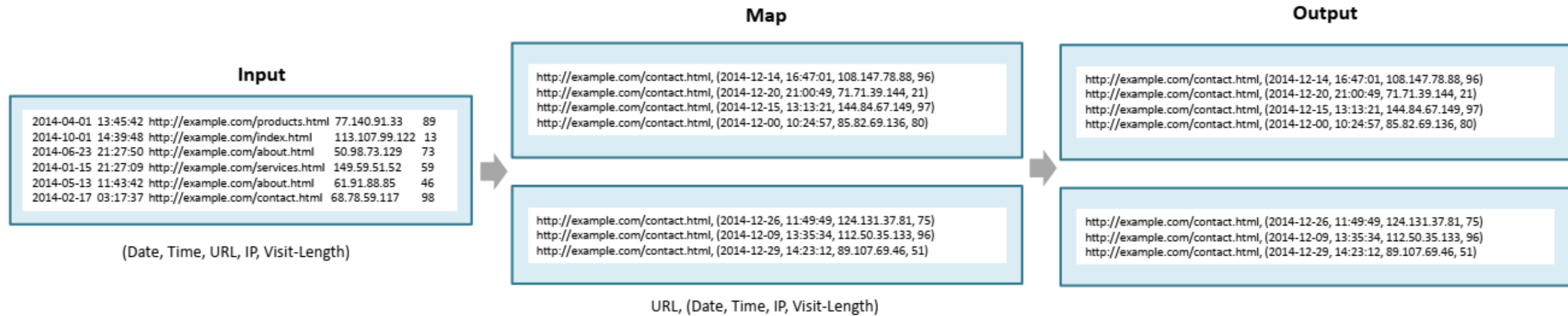
class MRmyjob(MRJob):
    def mapper(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()
        visit_len=int(data[4].strip())
        #Extract year from date
        year=date[0:4]
        month=date[5:7]

        #Emit (month,url) pair and 1 if year is 2014
        if year=='2014' and month=='12' and
           url=='http://example.com/contact.html':
            yield url, (date,time,ip,visit_len)

if __name__ == '__main__':
    MRmyjob.run()
```

Mẫu Filter



Mẫu Distinct

```
# Distint IP addresses

from mrjob.job import MRJob

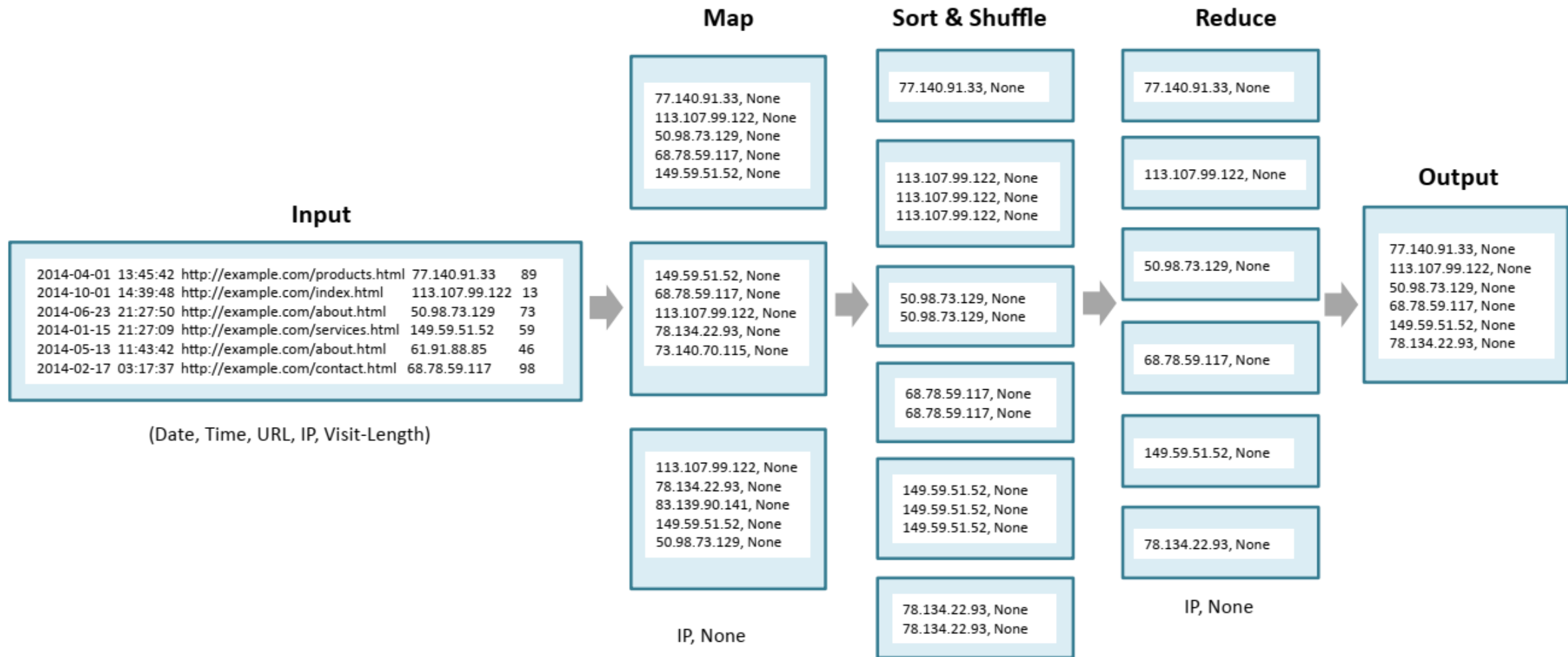
class MRmyjob(MRJob):
    def mapper(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()
        visit_printlen=int(data[4].strip())
        yield ip, None

    def reducer(self, key, list_of_values):
        yield key, None

if __name__ == '__main__':
    MRmyjob.run()
```

Mẫu Distinct



Mẫu Binning

```
# Partition records by Quarter

from mrjob.job import MRJob

class MRmyjob(MRJob):
    def mapper(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()
        visit_len=int(data[4].strip())

        #Extract year from date
        year=date[0:4]
        month=int(date[5:7])

        #Emit url and 1 if year is 2014
        if year=='2014':
            if month<=3:
                yield "Q1", (date, time, url, ip, visit_len)
            elif month<=6:
                yield "Q2", (date, time, url, ip, visit_len)
            elif month<=9:
                yield "Q3", (date, time, url, ip, visit_len)
            else:
                yield "Q4", (date, time, url, ip, visit_len)
```

Mẫu Binning

Input

2014-04-01	13:45:42	http://example.com/products.html	77.140.91.33	89
2014-10-01	14:39:48	http://example.com/index.html	113.107.99.122	13
2014-06-23	21:27:50	http://example.com/about.html	50.98.73.129	73
2014-01-15	21:27:09	http://example.com/services.html	149.59.51.52	59
2014-05-13	11:43:42	http://example.com/about.html	61.91.88.85	46
2014-02-17	03:17:37	http://example.com/contact.html	68.78.59.117	98

(Date, Time, URL, IP, Visit-Length)

Map

Q2, (2014-06-30, 10:48:26, http://example.com/products.html, 144.66.57.91, 38)
Q1, (2014-01-01, 16:49:60, http://example.com/products.html, 78.134.22.93, 95)
Q4, (2014-10-24, 18:24:46, http://example.com/contact.html, 83.139.90.141, 55)
Q3, (2014-08-02, 08:31:32, http://example.com/services.html, 73.140.70.115, 97)

Q4, (2014-10-21, 21:02:53, http://example.com/contact.html, 126.95.94.123, 22)
Q3, (2014-08-13, 21:49:01, http://example.com/products.html, 125.135.87.76, 87)
Q2, (2014-04-27, 08:51:05, http://example.com/services.html, 54.50.36.149, 45)
Q3, (2014-06-21, 09:27:56, http://example.com/about.html, 97.146.83.118, 46)

Quarter, (Date, Time, URL, IP, Visit-Length)

Output

Q1, (2014-01-01, 16:49:60, http://example.com/products.html, 78.134.22.93, 95)

Q2, (2014-06-30, 10:48:26, http://example.com/products.html, 144.66.57.91, 38)
Q2, (2014-04-27, 08:51:05, http://example.com/services.html, 54.50.36.149, 45)

Q3, (2014-08-02, 08:31:32, http://example.com/services.html, 73.140.70.115, 97)
Q3, (2014-08-13, 21:49:01, http://example.com/products.html, 125.135.87.76, 87)
Q3, (2014-06-21, 09:27:56, http://example.com/about.html, 97.146.83.118, 46)

Q4, (2014-10-24, 18:24:46, http://example.com/contact.html, 83.139.90.141, 55)
Q4, (2014-10-21, 21:02:53, http://example.com/contact.html, 126.95.94.123, 22)

Mẫu Inverted Index

- Lưu trữ các ánh xạ từ nội dung (từ trong một tài liệu hay Web) vào vị trí của nội dung (tên tập tin hay URL)
- Giúp tìm nhanh thông tin cho các bộ máy tìm kiếm Google.

```
#Inverted index

from mrjob.job import MRJob

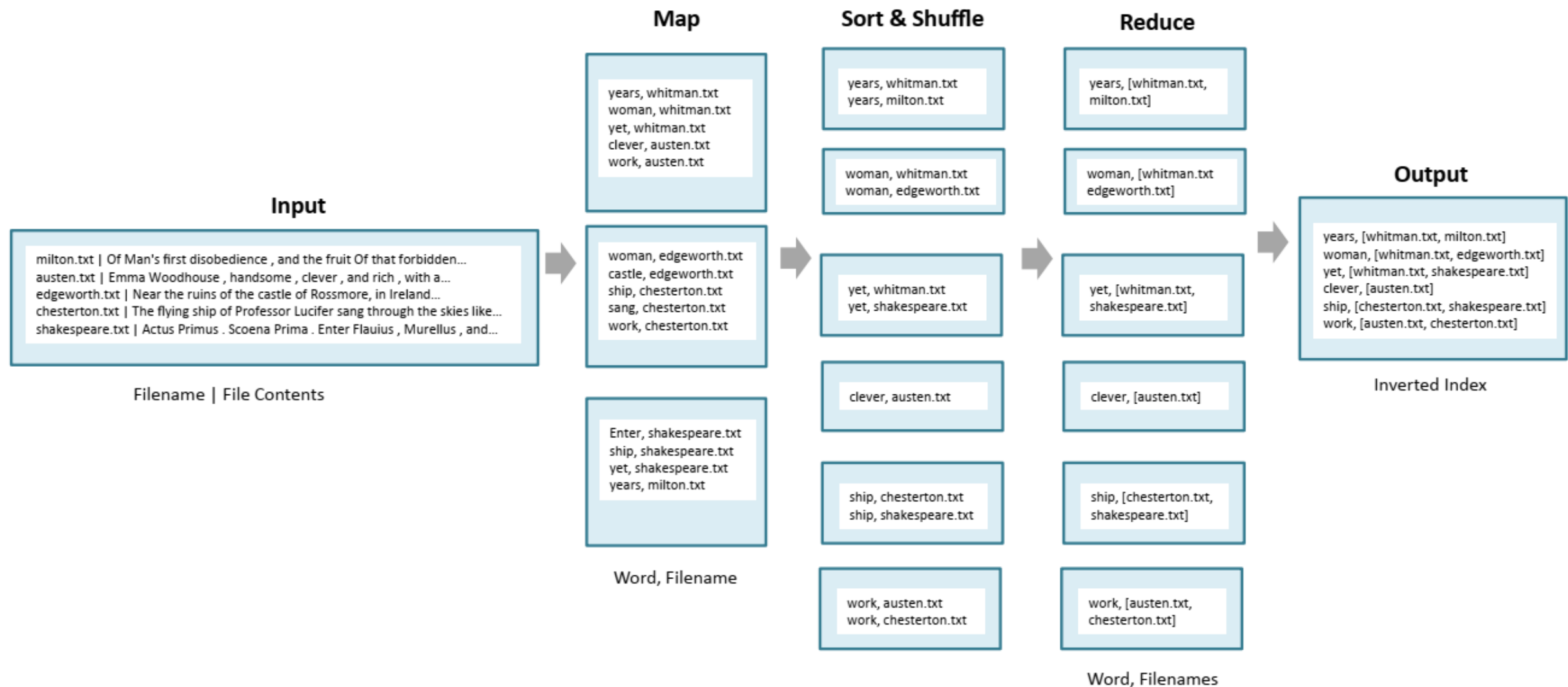
class MRmyjob(MRJob):
    def mapper(self, _, line):
        doc_id, content = line.split('|')
        words = content.split()
        for word in words:
            yield word, doc_id

    def reducer(self, key, list_of_values):
        docs=[]
        for x in list_of_values:
            docs.append(x)

        yield key, docs

if __name__ == '__main__':
    MRmyjob.run()
```

Mẫu Inverted Index



Mẫu Sort

```
# Sort by visit length

from mrjob.job import MRJob

class MRmyjob(MRJob):
    def mapper(self, _, line):
        #Split the line with tab separated fields
        data=line.split('\t')

        #Parse line
        date = data[0].strip()
        time = data[1].strip()
        url = data[2].strip()
        ip = data[3].strip()
        visit_len=int(data[4].strip())
        #Extract year from date
        year=date[0:4]
        month=date[5:7]

        #Emit url and 1 if year is 2014
        if year=='2014':
            yield None, (visit_len, (date, time, url, ip))

    def reducer(self, key, list_of_values):
        list_of_values = sorted(list(list_of_values), reverse=True)
        return list_of_values

if __name__ == '__main__':
    MRmyjob.run()
```

Mẫu Sort

Input

2014-04-01	13:45:42	http://example.com/products.html	77.140.91.33	89
2014-10-01	14:39:48	http://example.com/index.html	113.107.99.122	13
2014-06-23	21:27:50	http://example.com/about.html	50.98.73.129	73
2014-01-15	21:27:09	http://example.com/services.html	149.59.51.52	59
2014-05-13	11:43:42	http://example.com/about.html	61.91.88.85	46
2014-02-17	03:17:37	http://example.com/contact.html	68.78.59.117	98

(Date, Time, URL, IP, Visit-Length)

Map

None, (96, (2014-12-14, 16:47:01, http://example.com/contact.html, 108.147.78.88))
None, (21, (2014-12-20, 21:00:49, http://example.com/contact.html, 71.71.39.144))
None, (13, (2014-10-01, 14:39:48, http://example.com/index.html, 113.107.99.122))
None, (73, (2014-06-23, 21:27:50, http://example.com/about.html, 50.98.73.129))

None, (89, (2014-04-01, 13:45:42, http://example.com/products.html, 77.140.91.33))
None, (59, (2014-01-15, 21:27:09, http://example.com/services.html, 149.59.51.52))
None, (46, (2014-05-13, 11:43:42, http://example.com/about.html, 61.91.88.85))
None, (98, (2014-02-17, 03:17:37, http://example.com/contact.html, 68.78.59.117))

None, (Visit-Length, (Date, Time, URL, IP))

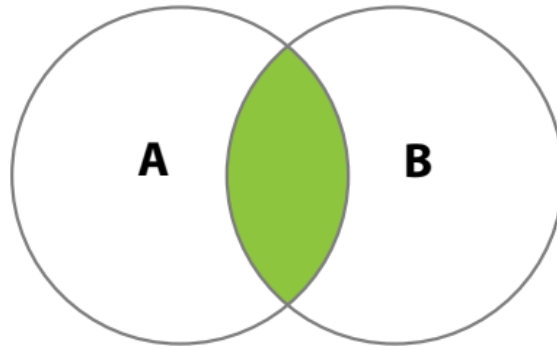
Reduce

98, (2014-02-17, 03:17:37, http://example.com/contact.html, 68.78.59.117)
96, (2014-12-14, 16:47:01, http://example.com/contact.html, 108.147.78.88)
89, (2014-04-01, 13:45:42, http://example.com/products.html, 77.140.91.33)
73, (2014-06-23, 21:27:50, http://example.com/about.html, 50.98.73.129)
59, (2014-01-15, 21:27:09, http://example.com/services.html, 149.59.51.52)
46, (2014-05-13, 11:43:42, http://example.com/about.html, 61.91.88.85)
21, (2014-12-20, 21:00:49, http://example.com/contact.html, 71.71.39.144)
13, (2014-10-01, 14:39:48, http://example.com/index.html, 113.107.99.122)

Visit-Length, (Date, Time, URL, IP)

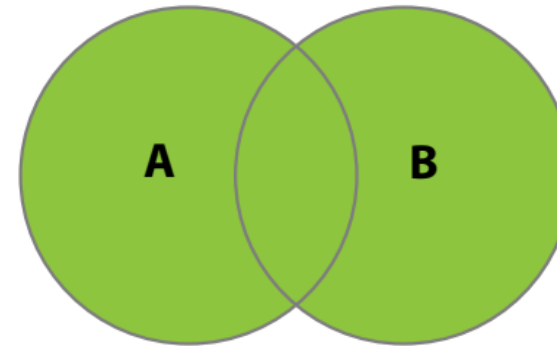
Mẫu Joins

Inner Join



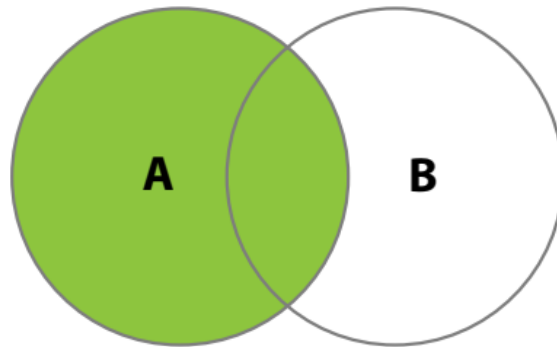
Returns rows from both tables which have the same value for the matching column or foreign key

Full Outer Join



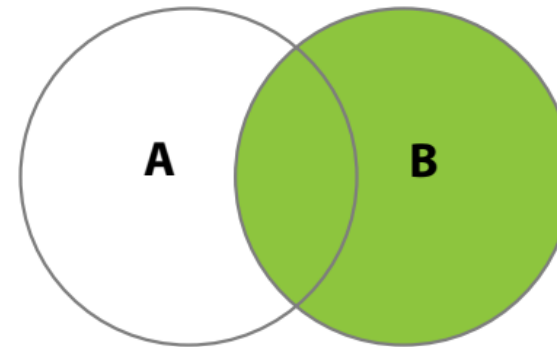
Returns all rows from both tables and returns NULL in columns of each table where no row matches

Left Outer Join



Returns all rows from table A and returns NULL in columns of table B where no row matches

Right Outer Join



Returns all rows from table B and returns NULL in columns of table A where no row matches



Mẫu Joins

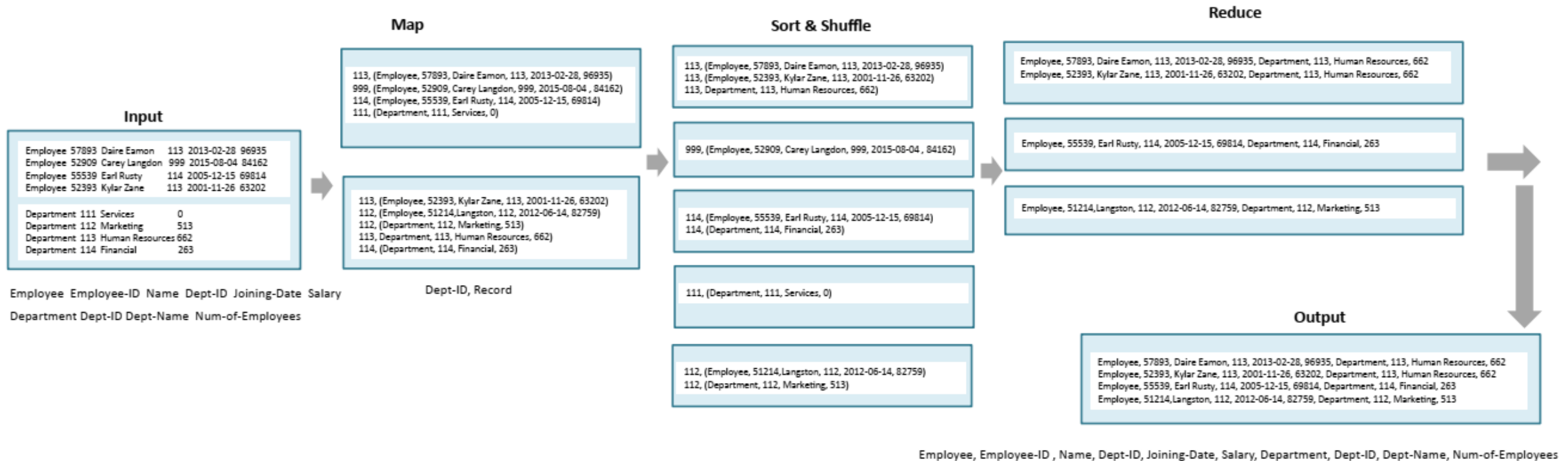
```
from mrjob.job import MRJob

class MyMRJob(MRJob):
    def mapper(self, _, line):
        data=line.split('^')
        if data[0]=='Employee':
            deptID = data[3]
        elif data[0]=='Department':
            deptID = data[1]
        yield deptID, data

    def reducer(self, key, list_of_values):
        values = list(list_of_values)
        employees = [ ]
        departments=[ ]
        for v in values:
            if v[0]=='Employee':
                employees.append(v)
            elif v[0]=='Department':
                departments.append(v)

        #Inner Join
        for e in employees:
            for d in departments:
                yield None, (e+d)
```

Mẫu Joins



Mẫu Joins

```
from mrjob.job import MRJob

class MyMRJob(MRJob):
    def mapper(self, _, line):
        data=line.split('^')
        if data[0]=='Employee':
            deptID = data[3]
        elif data[0]=='Department':
            deptID = data[1]
        yield deptID, data

    def reducer(self, key, list_of_values):
        values = list(list_of_values)
        employees = []
        departments=[]
        for v in values:
            if v[0]=='Employee':
                employees.append(v)
            elif v[0]=='Department':
                departments.append(v)

        #Left Outer Join
        for e in employees:
            if len(departments)>0:
                for d in departments:
                    yield None, (e+d)
            else:
                yield None, (e)
```

```
from mrjob.job import MRJob

class MyMRJob(MRJob):
    def mapper(self, _, line):
        data=line.split('^')
        if data[0]=='Employee':
            deptID = data[3]
        elif data[0]=='Department':
            deptID = data[1]
        yield deptID, data

    def reducer(self, key, list_of_values):
        values = list(list_of_values)
        employees = []
        departments=[]
        for v in values:
            if v[0]=='Employee':
                employees.append(v)
            elif v[0]=='Department':
                departments.append(v)

        #Full Outer Join
        if len(employees)>0:
            for e in employees:
                if len(departments)>0:
                    for d in departments:
                        yield None, (e+d)
                else:
                    yield None, (e)
        else:
            for d in departments:
                yield None, (d)
```



Bài tập

1. Word Count

- **Mô tả:** Đếm số lần xuất hiện của mỗi từ trong một bộ dữ liệu văn bản.
- input1.txt: Hello world, Hello Hadoop
- input2.txt: Hadoop MapReduce, Hello MapReduce

- **Kết quả:**

Hello: 3

world: 1

Hadoop: 2

MapReduce: 2

Bài tập

2. Inverted Index (Chỉ mục ngược)

- **Mô tả:** Xây dựng chỉ mục ngược cho các từ trong một bộ tài liệu.

- doc1.txt: MapReduce is great
- doc2.txt: Hadoop is powerful
- doc3.txt: MapReduce and Hadoop

- **Kết quả:**

MapReduce: doc1, doc3

is: doc1, doc2

great: doc1

Hadoop: doc2, doc3

powerful: doc2

and: doc3

Bài tập

3. Distributed Sorting

- **Mô tả:** Sắp xếp một tập dữ liệu lớn các số theo thứ tự tăng dần.
- numbers.txt: 5, 3, 8, 6, 2, 10, 4

Kết quả:

2, 3, 4, 5, 6, 8, 10

Bài tập

4. Join Operations (Phép nối)

- **Mô tả:** Thực hiện phép nối giữa hai tập dữ liệu, một tập chứa thông tin nhân viên và một tập chứa thông tin lương.

employees.txt (EmployeeID, Name):

1, John; 2, Jane; 3, Mark; 4, Lucy

salaries.txt (EmployeeID, Salary)

1, 5000; 2, 6000; 3, 5500; 4, 7000

Kết quả:

1, John, 5000; 2, Jane, 6000; 3, Mark, 5500; 4, Lucy, 7000



TRƯỜNG ĐẠI HỌC
VĂN LANG

Đạo đức - Ý chí - Sáng tạo

KHOA CÔNG NGHỆ THÔNG TIN

